EX 9 – Forward Chaining

Aim

To implement and demonstrate **Forward Chaining in Prolog** using simple inference, fire alert detection, and water purity detection.

Exercise 1: Simple Inference

Program:

```
None
:- dynamic known/1.

% Initial knowledge
start :-
    assert(known(a)),
    assert(known(b)),
    infer.

% Rule
infer :-
    known(a),
    known(b),
    \+ known(c),
    assert(known(c)),
    write('Inferred c'),
    nl.
```

Query:

```
None
?- start.
```

Output:

```
None
Inferred c
true.
```

Exercise 2: Fire Alert System

Program:

```
None
:- dynamic known/1.

% Rule to infer fire
check :-
    known(smoke),
    known(heat),
    assert(known(fire)),
    write('Fire Alert!'),
    nl.

% Start rule
start :-
    assert(known(smoke)),
    assert(known(heat)),
    check.
```

Query:

```
None
?- start.
```

Output:

```
None
Fire Alert!
true.
```

Exercise 3: Water Purity Detection

Program:

```
None
:- dynamic known/1.

% Rule to infer pollution
infer_pollution :-
    known(cloudy_water),
    known(bad_smell),
    assert(known(polluted)),
    write('Water is polluted'),
    nl.

start :-
    assert(known(cloudy_water)),
    assert(known(bad_smell)),
    infer_pollution.
```

Query:

```
None
?- start.
```

Output:

```
None
Water is polluted
true.
```

Result

Thus, the **Forward Chaining** technique was successfully implemented in Prolog. The system started with known facts and automatically derived new facts using the given rules.